Name:CH.Kruthankiran H.No:2303A51404 Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week8 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:16.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 16 – Database Design and Queries: Schema Design and SQL Generation**	Week8 - Wednesday

Lab Objectives:

- Understand schema design principles for relational databases.
- Use AI-assisted tools to generate **SQL schema definitions**.
- Write and refine **SQL queries** (CRUD + aggregate operations).
- Explore how AI can optimize queries and ensure normalization.

Task 1 – Hospital Management Database

Task:

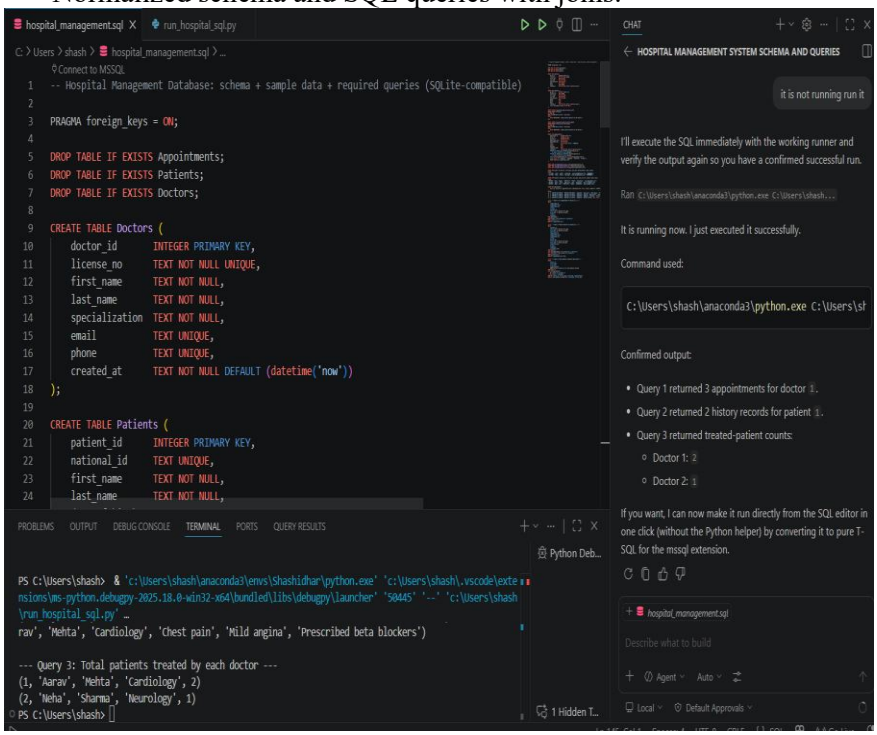
Create schema and queries for a **Hospital Management System**.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
  - List all appointments for a specific doctor.
  - Retrieve patient history by patient ID.
  - Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.



hospital_management.sql × run_hospital_sql.py

C: > Users > shash > hospital_management.sql > ...

```sql
CREATE TABLE Appointments (
    appointment_id      INTEGER PRIMARY KEY,
    doctor_id           INTEGER NOT NULL,
    patient_id          INTEGER NOT NULL,
    appointment_start   TEXT NOT NULL,
    appointment_end     TEXT NOT NULL,
    status              TEXT NOT NULL DEFAULT 'scheduled',
    reason              TEXT,
    diagnosis           TEXT,
    treatment_notes     TEXT,
    created_at          TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CHECK (datetime(appointment_end) > datetime(appointment_start)),
    CHECK (status IN ('scheduled', 'completed', 'cancelled', 'no_show')),
    UNIQUE (doctor_id, appointment_start)
);

CREATE INDEX idx_appointments_doctor_id ON Appointments(doctor_id);
CREATE INDEX idx_appointments_patient_id ON Appointments(patient_id);
CREATE INDEX idx_appointments_start ON Appointments(appointment_start);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Deb...

```
PS C:\Users\shash>  & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\exte
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '--' 'c:\Users\shash
\run_hospital_sql.py' …
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash>
```

1 Hidden T...

hospital_management.sql run_hospital_sql.py ×

C: > Users > shash > run_hospital_sql.py > ...

```python
import sqlite3
from pathlib import Path

sql_path = Path(r"C:\Users\shash\hospital_management.sql")

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

sql_text = sql_path.read_text(encoding="utf-8")

# Remove sqlite shell meta commands (e.g., .print) before executescript.
clean_lines = []
for line in sql_text.splitlines():
    if line.strip().startswith("."):
        continue
    clean_lines.append(line)

cur.executescript("\n".join(clean_lines))

print("--- Query 1: All appointments for doctor_id = 1 ---")
for row in cur.execute(
    """
    SELECT
        a.appointment_id,
        a.appointment_start,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Deb...

```
PS C:\Users\shash>  & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\exte
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '--' 'c:\Users\shash
\run_hospital_sql.py' …
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash>
```

1 Hidden T...

hospital_management.sql | run_hospital_sql.py

C: > Users > shash > run_hospital_sql.py > ...

```python
    SELECT
        a.appointment_id,
        a.appointment_start,
        a.appointment_end,
        a.status,
        p.patient_id,
        p.first_name AS patient_first_name,
        p.last_name  AS patient_last_name,
        a.reason,
        a.diagnosis
    FROM Appointments a
    JOIN Patients p ON p.patient_id = a.patient_id
    WHERE a.doctor_id = 1
    ORDER BY a.appointment_start;
    """
):
    print(row)

print("\n--- Query 2: Patient history for patient_id = 1 ---")
for row in cur.execute(
    """
    SELECT
        p.patient_id,
        p.first_name AS patient_first_name,
        p.last_name  AS patient_last_name,
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULTS

Python Deb...

```
PS C:\Users\shash>  & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\exte
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '--' 'c:\Users\shash
\run_hospital_sql.py' …
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
```

1 Hidden I...

hospital_management.sql | run_hospital_sql.py

C: > Users > shash > run_hospital_sql.py > ...

```python
        ORDER BY a.appointment_start DESC;
        """
    ):
        print(row)

print("\n--- Query 3: Total patients treated by each doctor ---")
for row in cur.execute(
        """
        SELECT
            d.doctor_id,
            d.first_name,
            d.last_name,
            d.specialization,
            COUNT(DISTINCT a.patient_id) AS total_patients_treated
        FROM Doctors d
        LEFT JOIN Appointments a
            ON a.doctor_id = d.doctor_id
           AND a.status = 'completed'
        GROUP BY d.doctor_id, d.first_name, d.last_name, d.specialization
        ORDER BY total_patients_treated DESC, d.last_name, d.first_name;
        """
    ):
        print(row)

conn.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '-
\run_hospital_sql.py' …
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash>
```

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an **E-commerce platform**.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest **normalization improvements** and **query optimization**.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
import sqlite3
from datetime import datetime

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    phone TEXT UNIQUE,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY,
    sku TEXT NOT NULL UNIQUE,
    product_name TEXT NOT NULL,
    unit_price NUMERIC NOT NULL CHECK (unit_price >= 0),
    is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Deb...

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '--' 'c:\Users\shash\AAC A 16.3.py'
Orders by user: 1
(4, '2026-04-02 09:25:00', 'paid', 6, 2, 'Mechanical Keyboard', 2, 80, 160)
(4, '2026-04-02 09:25:00', 'paid', 7, 3, 'USB-C Hub', 1, 45, 45)
(2, '2026-03-18 16:30:00', 'delivered', 3, 2, 'Mechanical Keyboard', 1, 80, 80)
(2, '2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25)
(1, '2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50)
```

1 Hidden T...

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

give code in python without comments

```python
import sqlite3
from datetime import datetime

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
```

AAC A 16.3.py

Describe what to build

Agent Auto

Local Default Approvals

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('pending','paid','shipped','delivered','cancelled','r
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price_at_order NUMERIC NOT NULL CHECK (unit_price_at_order >= 0),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES Products(product_id) ON UPDATE CASCADE ON DELETE RESTRIC
    UNIQUE (order_id, product_id)
);

CREATE INDEX idx_orders_user_date ON Orders(user_id, order_date);
CREATE INDEX idx_orderdetails_order ON OrderDetails(order_id);
CREATE INDEX idx_orderdetails_product ON OrderDetails(product_id);
CREATE INDEX idx_orders_date ON Orders(order_date);
"""
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Deb...

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '--' 'c:\Users\shash\AAC A 16.3.py' …
(2, '2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25)
(1, '2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50)
(1, '2026-03-05 10:00:00', 'paid', 2, 3, 'USB-C Hub', 1, 45, 45)

Most purchased product:
(1, 'Wireless Mouse', 6)
```

1 Hidden T...

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```
INSERT INTO Users (email, full_name, phone) VALUES
('alice@example.com', 'Alice Stone', '900000001'),
('bob@example.com', 'Bob Ray', '900000002');

INSERT INTO Products (sku, product_name, unit_price) VALUES
('SKU-100', 'Wireless Mouse', 25.00),
('SKU-101', 'Mechanical Keyboard', 80.00),
('SKU-102', 'USB-C Hub', 45.00);

INSERT INTO Orders (user_id, order_date, status) VALUES
(1, '2026-03-05 10:00:00', 'paid'),
(1, '2026-03-18 16:30:00', 'delivered'),
(2, '2026-03-20 12:10:00', 'shipped'),
(1, '2026-04-02 09:25:00', 'paid');

INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price_at_order) VALUES
(1, 1, 2, 25.00),
(1, 3, 1, 45.00),
(2, 2, 1, 80.00),
(2, 1, 1, 25.00),
(3, 1, 3, 25.00),
(4, 2, 2, 80.00),
(4, 3, 1, 45.00);
"""
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' '
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '
722' '--' 'c:\Users\shash\AAC A 16.3.py' …

Total revenue for 2026-03:
275

Normalization improvements:
- Move order status values to an OrderStatus lookup table and reference by status_id.
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
query_orders_by_user = """
SELECT
    o.order_id,
    o.order_date,
    o.status,
    od.order_detail_id,
    p.product_id,
    p.product_name,
    od.quantity,
    od.unit_price_at_order,
    (od.quantity * od.unit_price_at_order) AS line_total
FROM Orders o
JOIN OrderDetails od ON od.order_id = o.order_id
JOIN Products p ON p.product_id = od.product_id
WHERE o.user_id = ?
ORDER BY o.order_date DESC, o.order_id, od.order_detail_id;
"""

query_most_purchased_product = """
SELECT
    p.product_id,
    p.product_name,
    SUM(od.quantity) AS total_units_sold
FROM OrderDetails od
JOIN Products p ON p.product_id = od.product_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '5
722' '--' 'c:\Users\shash\AAC A 16.3.py' …
Normalization improvements:
- Move order status values to an OrderStatus lookup table and reference by status_id.
- Split Products into ProductCatalog and ProductPricingHistory for time-based price changes.
- Add Address table and keep shipping/billing addresses per order in OrderAddress.
- Create ProductCategory and ProductCategoryMap for many-to-many category assignments.
- Add Payment and Shipment tables to avoid overloading Orders with transactional details.
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
query_total_revenue_month = """
SELECT
    COALESCE(SUM(od.quantity * od.unit_price_at_order), 0) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON od.order_id = o.order_id
WHERE strftime('%Y-%m', o.order_date) = ?
  AND o.status IN ('paid', 'shipped', 'delivered');
"""

normalization_improvements = [
    "Move order status values to an OrderStatus lookup table and reference by status_id.",
    "Split Products into ProductCatalog and ProductPricingHistory for time-based price changes
    "Add Address table and keep shipping/billing addresses per order in OrderAddress.",
    "Create ProductCategory and ProductCategoryMap for many-to-many category assignments.",
    "Add Payment and Shipment tables to avoid overloading Orders with transactional details."
]

query_optimization_suggestions = [
    "Use composite index Orders(user_id, order_date) for user-order history queries.",
    "Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.",
    "Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) a
    "Use parameterized queries and prepared statements to improve plan reuse and safety.",
    "For high scale analytics, maintain a daily sales summary table via ETL or incremental job
]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '--' 'c:\Users\shash\AAC A 16.3.py' …
Query optimization suggestions:
- Use composite index Orders(user_id, order_date) for user-order history queries.
- Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.
- Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) and indexing i
t.
- Use parameterized queries and prepared statements to improve plan reuse and safety.
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def run_demo():

    print("Orders by user:", user_id)
    for row in orders:
        print(row)

    print("\nMost purchased product:")
    print(top_product)

    print("\nTotal revenue for", month_key + ":")
    print(revenue)

    print("\nNormalization improvements:")
    for item in normalization_improvements:
        print("-", item)

    print("\nQuery optimization suggestions:")
    for item in query_optimization_suggestions:
        print("-", item)

    conn.close()

if __name__ == "__main__":
    run_demo()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '--' 'c:\Users\shash\AAC A 16.3.py' …
- Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.
- Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) and indexing i
t.
- Use parameterized queries and prepared statements to improve plan reuse and safety.
- For high scale analytics, maintain a daily sales summary table via ETL or incremental jobs.
PS C:\Users\shash>
```

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
    - Retrieve all books currently issued.
    - Find overdue books (loan date > 30 days).
    - Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
import sqlite3

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Members;
DROP TABLE IF EXISTS Books;

CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY,
    isbn TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    category TEXT,
    published_year INTEGER,
    total_copies INTEGER NOT NULL CHECK (total_copies >= 0),
    available_copies INTEGER NOT NULL CHECK (available_copies >= 0 AND available_copies <= total_copies)
);

CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY,
    membership_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60
105' '--' 'c:\Users\shash\AAC A 16.3.py'
Indexing Strategy:
- Loans(member_id) for member-wise history and aggregates
- Loans(book_id) for book availability and issue tracking
- Loans(return_date, due_date) for issued/overdue filtering
- Loans(loan_date) for date-window analytics
- Books(title, author) for catalog search
```

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

```python
def run():
    conn = sqlite3.connect(":memory:")
    conn.executescript(schema_sql)
    conn.executescript(seed_sql)

    print("Indexing Strategy:")
    for s in indexing_strategy:
        print("-", s)

    print("\nAll books currently issued:")
    for row in conn.execute(query_currently_iss
        print(row)

    print("\nOverdue books (loan date > 30 days
    for row in conn.execute(query_overdue_30_da
        print(row)

    print("\nBooks loaned by each member:")
    for row in conn.execute(query_books_loaned_
        print(row)

    conn.close()

if __name__ == "__main__":
    run()
```

AAC A 16.3.py

Describe what to build

Agent Auto

Local Default Approvals

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
    email TEXT UNIQUE,
    phone TEXT,
    joined_date TEXT NOT NULL
);

CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date TEXT NOT NULL,
    due_date TEXT NOT NULL,
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES Books(book_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (member_id) REFERENCES Members(member_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CHECK (date(due_date) >= date(loan_date)),
    CHECK (return_date IS NULL OR date(return_date) >= date(loan_date))
);

CREATE INDEX idx_loans_book_id ON Loans(book_id);
CREATE INDEX idx_loans_member_id ON Loans(member_id);
CREATE INDEX idx_loans_return_due ON Loans(return_date, due_date);
CREATE INDEX idx_loans_loan_date ON Loans(loan_date);
CREATE INDEX idx_books_title_author ON Books(title, author);
CREATE INDEX idx_members_name ON Members(full_name);
"""
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60
105' '--' 'c:\Users\shash\AAC A 16.3.py' …
All books currently issued:
(5, 2, '1984', 'George Orwell', 3, 'Nina Paul', '2026-03-27', '2026-04-26')
(1, 1, 'The Odyssey', 'Homer', 1, 'Asha Rao', '2026-03-20', '2026-04-19')
(3, 3, 'The Great Gatsby', 'F. Scott Fitzgerald', 2, 'Kiran Das', '2026-03-12', '2026-04-11')
(2, 2, '1984', 'George Orwell', 1, 'Asha Rao', '2026-02-20', '2026-03-22')
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
seed_sql = """
INSERT INTO Books (isbn, title, author, category, published_year, total_copies, available_copies
('978-0140449136', 'The Odyssey', 'Homer', 'Epic', -700, 5, 2),
('978-0451524935', '1984', 'George Orwell', 'Dystopian', 1949, 4, 1),
('978-0743273565', 'The Great Gatsby', 'F. Scott Fitzgerald', 'Classic', 1925, 3, 0),
('978-0061120084', 'To Kill a Mockingbird', 'Harper Lee', 'Classic', 1960, 6, 4);

INSERT INTO Members (membership_no, full_name, email, phone, joined_date) VALUES
('M-1001', 'Asha Rao', 'asha@example.com', '9001111111', '2025-01-10'),
('M-1002', 'Kiran Das', 'kiran@example.com', '9002222222', '2025-03-22'),
('M-1003', 'Nina Paul', 'nina@example.com', '9003333333', '2025-06-05');

INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES
(1, 1, date('now','-12 day'), date('now','+18 day'), NULL),
(2, 1, date('now','-40 day'), date('now','-10 day'), NULL),
(3, 2, date('now','-20 day'), date('now','+10 day'), NULL),
(4, 3, date('now','-50 day'), date('now','-20 day'), date('now','-5 day')),
(2, 3, date('now','-5 day'), date('now','+25 day'), NULL);
"""

query_currently_issued = """
SELECT
    l.loan_id,
    b.book_id,
    b.title,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python De

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60
105' '--' 'c:\Users\shash\AAC A 16.3.py' …
Overdue books (loan date > 30 days):
(2, '1984', 'Asha Rao', '2026-02-20', '2026-03-22', 40)

Books loaned by each member:
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
```

1 Hidden

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```
    l.loan_id,
    b.book_id,
    b.title,
    b.author,
    m.member_id,
    m.full_name,
    l.loan_date,
    l.due_date
FROM Loans l
JOIN Books b ON b.book_id = l.book_id
JOIN Members m ON m.member_id = l.member_id
WHERE l.return_date IS NULL
ORDER BY l.loan_date DESC;
"""

query_overdue_30_days = """
SELECT
    l.loan_id,
    b.title,
    m.full_name,
    l.loan_date,
    l.due_date,
    CAST(julianday('now') - julianday(l.loan_date) AS INTEGER) AS days_since_loan
FROM Loans l
JOIN Books b ON b.book_id = l.book_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\pythor
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laur
105' '--' 'c:\Users\shash\AAC A 16.3.py' …

Books loaned by each member:
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
(2, 'Kiran Das', 1)
PS C:\Users\shash>
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def run():
    conn.executescript(schema_sql)
    conn.executescript(seed_sql)

    print("Indexing Strategy:")
    for s in indexing_strategy:
        print("-", s)

    print("\nAll books currently issued:")
    for row in conn.execute(query_currently_issued):
        print(row)

    print("\nOverdue books (loan date > 30 days):")
    for row in conn.execute(query_overdue_30_days):
        print(row)

    print("\nBooks loaned by each member:")
    for row in conn.execute(query_books_loaned_by_member):
        print(row)

    conn.close()

if __name__ == "__main__":
    run()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bur
105' '--' 'c:\Users\shash\AAC A 16.3.py' …

Books loaned by each member:
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
(2, 'Kiran Das', 1)
PS C:\Users\shash>
```

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

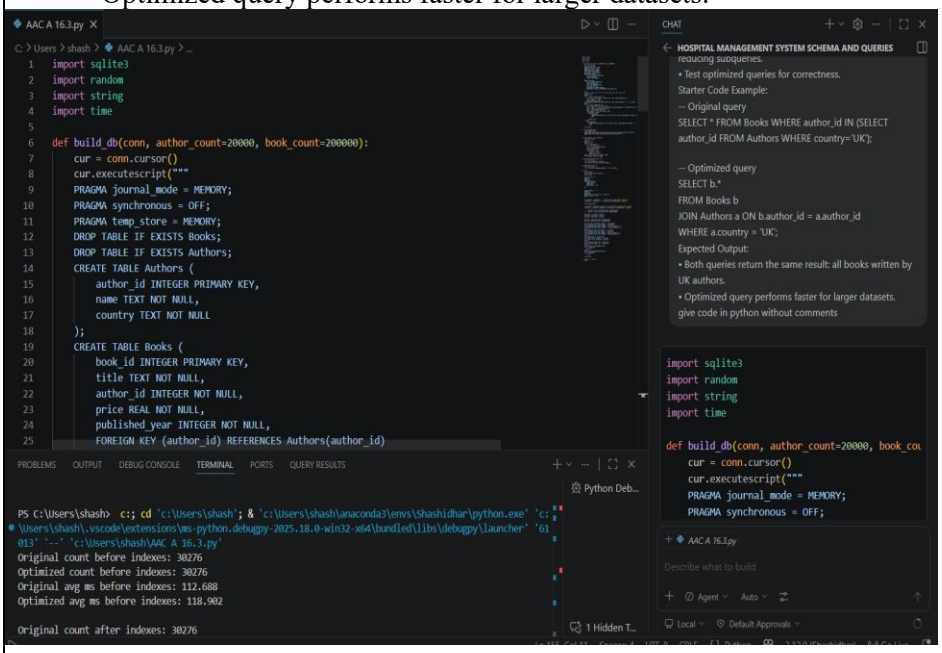
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

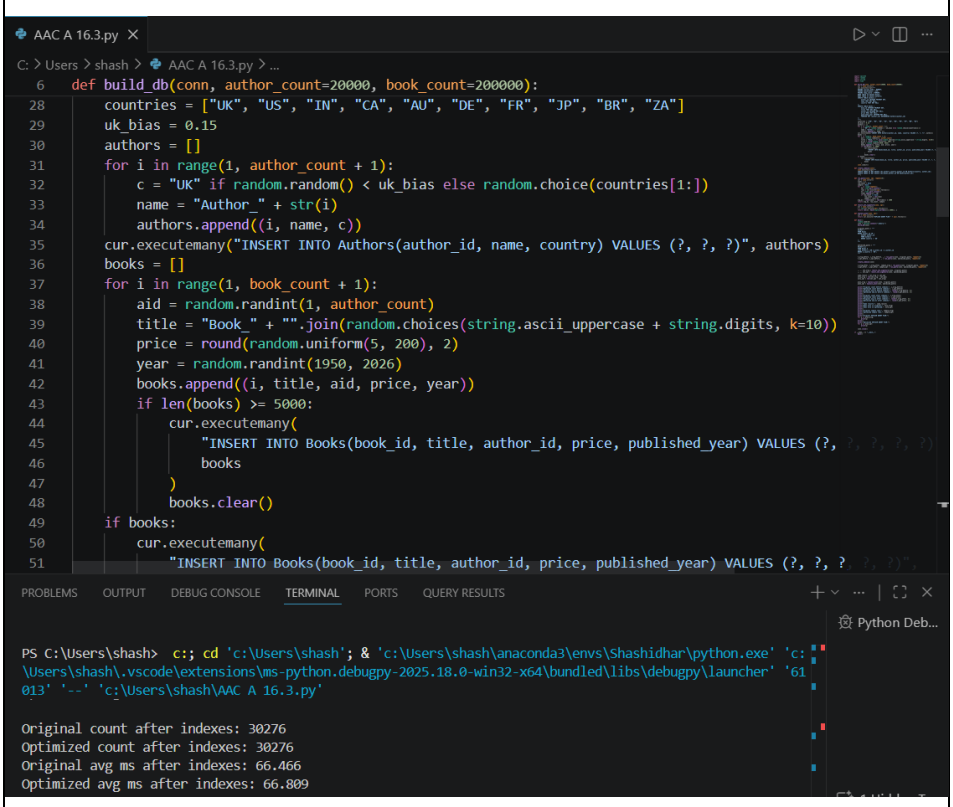
-- Optimized query

SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.





AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def create_indexes(conn):
    conn.executescript("""
    CREATE INDEX IF NOT EXISTS idx_authors_country_author_id ON Authors(country, author_id);
    CREATE INDEX IF NOT EXISTS idx_books_author_id ON Books(author_id);
    """)

def run_query(conn, sql, repeats=5):
    cur = conn.cursor()
    times = []
    row_count = None
    sample = None
    for _ in range(repeats):
        t0 = time.perf_counter()
        rows = cur.execute(sql).fetchall()
        t1 = time.perf_counter()
        times.append(t1 - t0)
        if row_count is None:
            row_count = len(rows)
            sample = rows[:5]
    avg_ms = (sum(times) / len(times)) * 1000
    return avg_ms, row_count, sample

def result_set_signature(conn, sql):
    cur = conn.cursor()
    s = set(cur.execute(sql).fetchall())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61
013' '--' 'c:\Users\shash\AAC A 16.3.py'

Same results: True
Rows only in original: 0
Rows only in optimized: 0

Original sample rows: [(7614, 'Book_VTJPLZQ9RA', 5, 147.19, 1981), (18739, 'Book_ZGE9F33KSH', 5, 37.09,
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def main():
    SELECT *
    FROM Books
    WHERE author_id IN (
        SELECT author_id
        FROM Authors
        WHERE country = 'UK'
    );
    """

    optimized_query = """
    SELECT b.*
    FROM Books b
    JOIN Authors a ON b.author_id = a.author_id
    WHERE a.country = 'UK';
    """

    t_orig_before, c_orig_before, _ = run_query(conn, original_query, repeats=3)
    t_opt_before, c_opt_before, _ = run_query(conn, optimized_query, repeats=3)

    create_indexes(conn)

    t_orig_after, c_orig_after, sample_orig = run_query(conn, original_query, repeats=5)
    t_opt_after, c_opt_after, sample_opt = run_query(conn, optimized_query, repeats=5)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Del

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61
013' '--' 'c:\Users\shash\AAC A 16.3.py'
2003), (54069, 'Book_61S15YZ6AO', 5, 137.3, 1972), (55424, 'Book_W6E6ADOE2F', 5, 126.16, 1966), (57441,
'Book_9CM28GE3EU', 5, 28.14, 2012)]
Optimized sample rows: [(7614, 'Book_VTJPLZQ9RA', 5, 147.19, 1981), (18739, 'Book_ZGE9F33KSH', 5, 37.09,
 2003), (54069, 'Book_61S15YZ6AO', 5, 137.3, 1972), (55424, 'Book_W6E6ADOE2F', 5, 126.16, 1966), (57441,
 'Book_9CM28GE3EU', 5, 28.14, 2012)]
```

1 Hidden

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def main():
    same_result = set_orig == set_opt
    only_orig = len(set_orig - set_opt)
    only_opt = len(set_opt - set_orig)

    plan_orig = explain_plan(conn, original_query)
    plan_opt = explain_plan(conn, optimized_query)

    print("Original count before indexes:", c_orig_before)
    print("Optimized count before indexes:", c_opt_before)
    print("Original avg ms before indexes:", round(t_orig_before, 3))
    print("Optimized avg ms before indexes:", round(t_opt_before, 3))
    print()
    print("Original count after indexes:", c_orig_after)
    print("Optimized count after indexes:", c_opt_after)
    print("Original avg ms after indexes:", round(t_orig_after, 3))
    print("Optimized avg ms after indexes:", round(t_opt_after, 3))
    print()
    print("Same results:", same_result)
    print("Rows only in original:", only_orig)
    print("Rows only in optimized:", only_opt)
    print()
    print("Original sample rows:", sample_orig)
    print("Optimized sample rows:", sample_opt)
    print()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.ex
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launche
013' '--' 'c:\Users\shash\AAC A 16.3.py'

Original EXPLAIN QUERY PLAN:
(3, 0, 0, 'SEARCH Books USING INDEX idx_books_author_id (author_id=?)')
(7, 0, 0, 'LIST SUBQUERY 1')
(9, 7, 0, 'SEARCH Authors USING COVERING INDEX idx_authors_country_author_id (country=?)')
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def main():
    print("Same results:", same_result)
    print("Rows only in original:", only_orig)
    print("Rows only in optimized:", only_opt)
    print()
    print("Original sample rows:", sample_orig)
    print("Optimized sample rows:", sample_opt)
    print()
    print("Original EXPLAIN QUERY PLAN:")
    for p in plan_orig:
        print(p)
    print()
    print("Optimized EXPLAIN QUERY PLAN:")
    for p in plan_opt:
        print(p)

    conn.close()

if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\pytho
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\lau
013' '--' 'c:\Users\shash\AAC A 16.3.py'
(9, 7, 0, 'SEARCH Authors USING COVERING INDEX idx_authors_country_author_id (country=?)')

Optimized EXPLAIN QUERY PLAN:
(4, 0, 0, 'SEARCH a USING COVERING INDEX idx_authors_country_author_id (country=?)')
(8, 0, 0, 'SEARCH b USING INDEX idx_books_author_id (author_id=?)')
PS C:\Users\shash>
```

Task 5: University Course Registration

Scenario:

SR University needs a **course registration system** for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
  - Each student can register for multiple courses.
  - Each course is taught by a faculty member.
- Write SQL queries to:
  - List all students enrolled in a specific course.
  - Find faculty members teaching more than 2 courses.
  - Retrieve courses with the highest number of registrations.

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
import sqlite3

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;

CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    roll_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL,
    joined_year INTEGER NOT NULL
);

CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    emp_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57
630' '--' 'c:\Users\shash\AAC A 16.3.py'
Students enrolled in CS101
('CS101', 'Database Systems', 1, 'SRU23CSE001', 'Aarav Reddy', '2026-Spring')
('CS101', 'Database Systems', 2, 'SRU23CSE002', 'Isha Kapoor', '2026-Spring')
('CS101', 'Database Systems', 3, 'SRU23ECE001', 'Karan Das', '2026-Spring')
('CS101', 'Database Systems', 4, 'SRU22CSE010', 'Meera Nair', '2026-Spring')
('CS101', 'Database Systems', 5, 'SRU22ECE005', 'Nikhil Varma', '2026-Spring')
```

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

Scenario:
SR University needs a course registration system for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
- Each student can register for multiple courses.
- Each course is taught by a faculty member.
- Write SQL queries to:
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.

give code in python without comments

```python
import sqlite3

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;

CREATE TABLE Students (
```

AAC A 16.3.py

Describe what to build

Agent Auto

Local Default Approvals

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_code TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    credits INTEGER NOT NULL CHECK (credits > 0),
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    semester TEXT NOT NULL,
    registered_at TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON UPDATE CASCADE ON DELETE RESTRIC
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    UNIQUE (student_id, course_id, semester)
);

CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
CREATE INDEX idx_registrations_student_id ON Registrations(student_id);
CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
"""
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57
630' '--' 'c:\Users\shash\AAC A 16.3.py'
Students enrolled in CS101
('CS101', 'Database Systems', 1, 'SRU23CSE001', 'Aarav Reddy', '2026-Spring')
('CS101', 'Database Systems', 2, 'SRU23CSE002', 'Isha Kapoor', '2026-Spring')
('CS101', 'Database Systems', 3, 'SRU23ECE001', 'Karan Das', '2026-Spring')
('CS101', 'Database Systems', 4, 'SRU22CSE010', 'Meera Nair', '2026-Spring')
('CS101', 'Database Systems', 5, 'SRU22ECE005', 'Nikhil Varma', '2026-Spring')
```

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

```python
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    roll_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL,
    joined_year INTEGER NOT NULL
);

CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    emp_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_code TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    credits INTEGER NOT NULL CHECK (credits > 0
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty
);

CREATE TABLE Registrations (
```

AAC A 16.3.py

Describe what to build

Agent Auto

Local Default Approvals

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```
CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
CREATE INDEX idx_registrations_student_id ON Registrations(student_id);
CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
"""

seed_sql = """
INSERT INTO Faculty (emp_no, full_name, email, department) VALUES
('F1001', 'Dr. Priya Sharma', 'priya.sharma@sru.edu', 'CSE'),
('F1002', 'Dr. Rahul Menon', 'rahul.menon@sru.edu', 'CSE'),
('F1003', 'Dr. Ananya Rao', 'ananya.rao@sru.edu', 'ECE');

INSERT INTO Courses (course_code, title, credits, faculty_id) VALUES
('CS101', 'Database Systems', 4, 1),
('CS102', 'Data Structures', 4, 1),
('CS201', 'Operating Systems', 4, 1),
('CS202', 'Machine Learning', 3, 2),
('EC101', 'Digital Electronics', 4, 3);

INSERT INTO Students (roll_no, full_name, email, department, joined_year) VALUES
('SRU23CSE001', 'Aarav Reddy', 'aarav.reddy@sru.edu', 'CSE', 2023),
('SRU23CSE002', 'Isha Kapoor', 'isha.kapoor@sru.edu', 'CSE', 2023),
('SRU23ECE001', 'Karan Das', 'karan.das@sru.edu', 'ECE', 2023),
('SRU22CSE010', 'Meera Nair', 'meera.nair@sru.edu', 'CSE', 2022),
('SRU22ECE005', 'Nikhil Varma', 'nikhil.varma@sru.edu', 'ECE', 2022);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe'
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
630' '--' 'c:\Users\shash\AAC A 16.3.py'

Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)

Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```
INSERT INTO Registrations (student_id, course_id, semester) VALUES
(1, 1, '2026-Spring'),
(1, 2, '2026-Spring'),
(1, 4, '2026-Spring'),
(2, 1, '2026-Spring'),
(2, 3, '2026-Spring'),
(3, 5, '2026-Spring'),
(3, 1, '2026-Spring'),
(4, 1, '2026-Spring'),
(4, 2, '2026-Spring'),
(4, 3, '2026-Spring'),
(5, 5, '2026-Spring'),
(5, 1, '2026-Spring');
"""

query_students_in_course = """
SELECT
    c.course_code,
    c.title,
    s.student_id,
    s.roll_no,
    s.full_name,
    r.semester
FROM Registrations r
JOIN Students s ON s.student_id = r.student_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Sha
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
630' '--' 'c:\Users\shash\AAC A 16.3.py'
Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)

Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

AAC A 16.3.py

C: > Users > shash > AAC A 16.3.py > ...

```python
def run():
    conn = sqlite3.connect(":memory:")
    conn.executescript(schema_sql)
    conn.executescript(seed_sql)

    course_code = "CS101"

    print("Students enrolled in", course_code)
    for row in conn.execute(query_students_in_course, (course_code,)):
        print(row)

    print("\nFaculty teaching more than 2 courses")
    for row in conn.execute(query_faculty_more_than_2_courses):
        print(row)

    print("\nCourses with highest number of registrations")
    for row in conn.execute(query_highest_registrations):
        print(row)

    conn.close()

if __name__ == "__main__":
    run()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash>  c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidh
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\deb
630' '--' 'c:\Users\shash\AAC A 16.3.py'
Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)

Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.